```
    [mday] => 14
    [wday] => 6
    ...
```

Most of the array keys are self-explanatory; `mday` is the numeric day of the month, and `wday` is the numeric day of the week (*0* is Sunday, *1* is Monday, and so on).

mktime(), gmmktime()

We've seen how to use `DateTime::getTimestamp()` to obtain a Unix timestamp, which of course requires instantiating a new `DateTime` object before you can invoke the method. An alternative approach is to use the `mktime()` or `gmmktime()` functions. These functions accept multiple integer arguments for `hour`, `minute`, `second`, `month`, `day`, and `year`, in that order (note the slightly odd ordering of day and month). The default values for any omitted argument are simply whatever is current for the system, as the following demonstrates:

```
$format = 'Y-m-d H:i:s';

$tstamp1 = mktime(0);
$tstamp2 = mktime(0, 0);
$tstamp3 = mktime(0, 0, 0);

print date($format, $tstamp1) . PHP_EOL;
print date($format, $tstamp2) . PHP_EOL;
print date($format, $tstamp3);

Output:
2025-01-03 00:47:28
2025-01-03 00:00:28
2025-01-03 00:00:00
```

`mktime()` *must* have at least one argument (hours) – we've used zero – since PHP 8.0. Prior versions of PHP allow for no arguments, and `mktime()` would behave like the `time()` function. PHP 8.0+ still allows the other `mktime()` arguments to be null, and in such cases the system's current time and date values are used:

```
// assuming current system date/time is 2025-01-03 15:47:25
$tstamp = mktime(
    hour: 0,        // overrides 15
```